A.I.S.S.M.S
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

Experiment - 04

- **AIM**: Write a Cuda program for -
  1. Addition of two large vectors.
  2. Matrix multiplication using CUDA C.

- **THEORY**:

(1) **Introduction to CUDA -**

- CUDA (compute unified device architecture) is a parallel computing platform and programming model developed by NVIDIA that allows developers to use graphics processing units (GPUs) for general purpose computing

- Unlike CPUs, which typically execute a small number of threads sequentially or with limited parallelism. , GPU consists of thousands of lightweight cores capable of executing massively parallel workloads efficiently.

- CUDA extends effectively the C/C++ programming language by introducing -
  - Kernel functions that run on the GPU.
  - Thread heirarchy (threads, blocks, grids).
  - Explicit memory management between host (CPU) and device (GPU).

A.I.S.S.M.S
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

CUDA is widely used in applications such as scientific computing, machine learning, image processing and high performance computing simulations due to its ability to significantly reduce execution time for data-parallel tasks.

	Scal	Vect	Cor	Car.
Hardware →	AL ALU	ALU	Simd	S S Gp.
Thread →				
	Thread	War	Thread block.	Block grid.
Memory →	Register	Register	ck	d
Address → Space.	Local per thread.	Local per thread.	Shared Memory	Glob.

Architecture diagram –

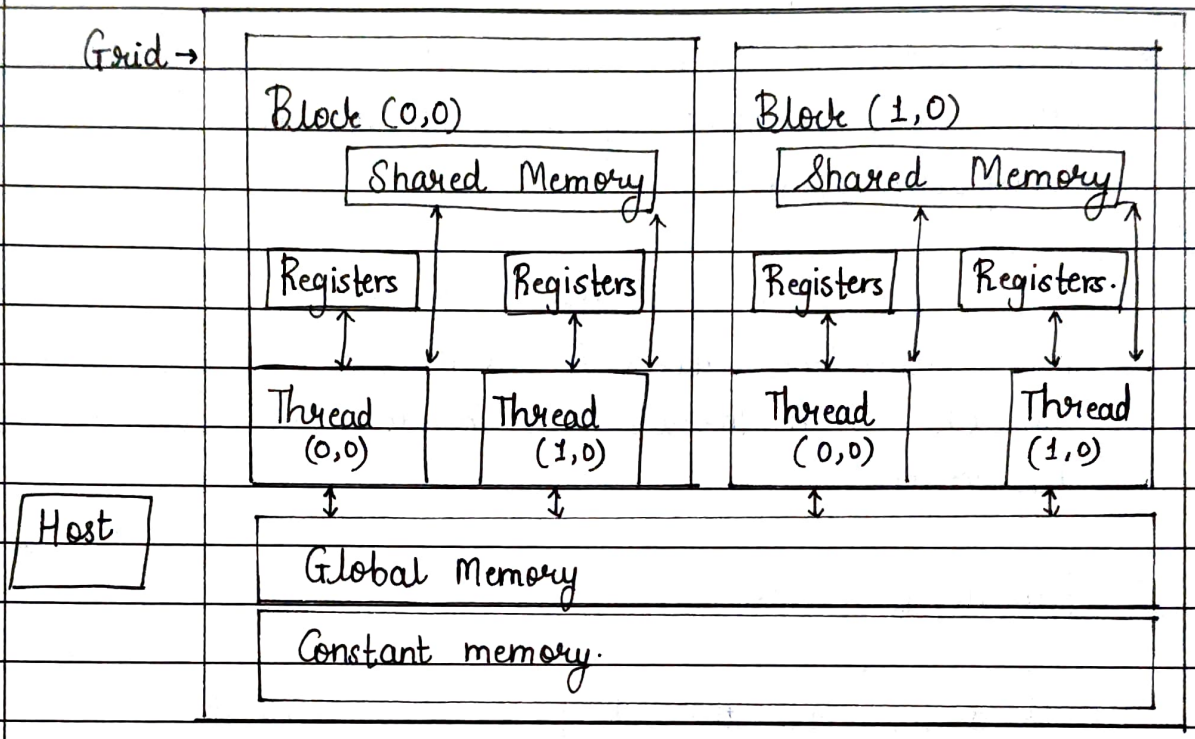
Host and device –
- Host : CPU and its memory.
- Device : GPU and its memory.

The programmer explicitly :-

1. Allocates memory on the GPU.
2. Transfers data from host to device.
3. Launches kernel functions.

A.I.S.S.M.S
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

4. Copies results back to host.



- **Thread Heirarchy –**

Cuda uses heirarchical execution model –

- Thread : Executes one instance of a Kernel.
- Block : Group of threads that can co-operate using shared memory.
- Grid : Collection of Blocks executing the same Kernel.

- **Memory Model.**

1. Global memory : Large but slow.
2. Shared memory : Fast, shared within a block.
3. Registers : Fastest, private to each thread.

A.I.S.S.M.S
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

(2). Problem 1: Addition of Two large vectors using CUDA.

- Vector addition is a classic data-parallel problem where each element of the output vector is computed independently as:

$$C[i] = A[i] + B[i].$$

This independence makes it ideal for GPU execution as each CUDA Thread can handle one element of the vectors.

- Parallization strategy -

1. Assign one thread per vector element.
2. Use global thread index to map threads to vector positions.
3. Launch enough threads to cover all elements.

- Observations -
  - Each thread computes one addition
  - Boundary check ensures safety.
  - High scalability for very large vectors.

- Procedure -

(1) Write a program using text editor, name the source code with .cu extension.
(2). Compile the program using nvcc compiler.
(3) Execute the program.
(4) Verify the result.

- The following steps are involved in implementing vector addition using CUDA C: Allocate memory on the device (GPU) using CUDA malloc().
- Copy input vectors from host (CPU) to device using cudaMemcpy()
- Launch the Kernel on the GPU using the syntax.
- Wait for the Kernel to finish using cudaDeviceSynchronize()
- Copy the result back from device to host using cudaMemcpy().
- Free memory on the device using cudafree().

(3) Matrix Multiplication -

- It is a fundamental operation in linear algebra that involves multiplying two matrices to produce a third matrix.
- It is a computationally intensive operation that can benefit from parallelization on GPUs.

$$C[i][j] = \sum_{K=0}^{N-1} A[i][k] \times B[k][j].$$

- Parallelization strategy -
  - Each thread computes one element of the output matrix.
  - Threads are organized in 2D grids and blocks.
  - Thread indices map directly to matrix row and column indices.

A.I.S.S.M.S
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

- Observations -

- Each thread computes one matrix element.
- Uses 2D thread indexing.
- Can be further optimized using shared memory tiling.

- **Conclusion -**

CUDA enables efficient parallel execution of computationally intensive tasks by exploiting the massive parallelism offered by GPUs. Vector addition demonstrates simple data parallelism, while matrix multiplication illustrates structured parallel computation using multidimensional grids. These examples highlight how CUDA significantly improves performance for large scale numerical operations compared to sequential CPU execution, making it a powerful tool for high performance computing applications.

* * * * * * * *.